🌿 EcoTrace – Backend, Database, Authentication, API & Security Architecture (v1.0)

System Vision

Build EcoTrace as a production-ready climate-tech platform capable of:

- 1M+ users
- Real-time carbon tracking
- Secure authentication
- AI-powered recommendations
- Social features
- Analytics
- Future integrations (Smart Homes, Wearables, Receipts)

Architecture Style:

Microservice-inspired Modular Monolith

Why?

Because:

Pure microservices are overkill for an MVP.

A modular monolith can easily transition to microservices later.

Recommended Tech Stack

Frontend

Next.js TypeScript TailwindCSS Shadcn UI Framer Motion

Backend

Node.js TypeScript Express.js or NestJS

Recommendation:

NestJS

Reason:

- Clean architecture
- Dependency Injection
- Better scalability
- Built-in validation
- Better security practices

Database

Primary Database: PostgreSQL

Cache: Redis

Search: PostgreSQL Full Text Search

Analytics: ClickHouse (future)

ORM

Prisma ORM

Benefits:

- Type-safe queries
- Automatic migrations
- Better developer experience
- Easy relationships

Cloud Infrastructure

Production:

Frontend: Vercel

Backend: AWS ECS AWS EC2 Docker

Database: AWS RDS PostgreSQL

Cache: Redis Cloud

Storage: AWS S3

CDN: Cloudflare

Email: Resend

Monitoring: Sentry Grafana Prometheus

Logs: Loki

CI/CD: GitHub Actions

High-Level Architecture

User ↓

CDN ↓

Next.js Frontend ↓

API Gateway ↓

Authentication Service ↓

Application Service ↓

PostgreSQL Redis S3

↓

Analytics Service ↓

Notification Service

↓

AI Recommendation Engine

Backend Modules

Auth Module

Responsibilities:

- Register
- Login
- Logout
- Refresh Tokens
- Password Reset
- Email Verification
- Social Login
- Session Management

User Module

Responsibilities:

- User Profile
- Preferences
- Goals
- Settings
- Privacy Controls
- Carbon Statistics

Carbon Module

Responsibilities:

- Onboarding Quiz
- Carbon Calculations
- Daily Logs
- Trends
- Score Generation

Tracker Module

Responsibilities:

- Daily Actions

- Streak System
- Badges
- Gamification

Insights Module

Responsibilities:

- Personalized Recommendations
- AI Suggestions
- Savings Calculations

Leaderboard Module

Responsibilities:

- Rankings
- Green Score
- Community Challenges

Notification Module

Responsibilities:

- Emails
- Push Notifications
- Weekly Reports
- Goal Reminders

Analytics Module

Responsibilities:

- User Activity
- Retention
- Engagement Metrics
- Carbon Impact Metrics

Database Schema

Users Table

users

id UUID PK name email password_hash avatar country timezone email_verified provider created_at updated_at

---

User Preferences

user_preferences

id UUID PK user_id FK theme notification_enabled public_profile measurement_system

---

Carbon Profile

carbon_profiles

id UUID PK user_id FK transport_score diet_score home_score flight_score shopping_score annual_total global_percentile updated_at

---

Daily Logs

daily_logs

id UUID PK user_id FK action_type category co2_delta logged_at

---

Streaks

streaks

id UUID PK user_id FK current_streak best_streak last_logged_day

---

Goals

goals

id UUID PK user_id FK target_reduction current_progress deadline status

---

User Badges

user_badges

id UUID PK user_id FK badge_id earned_at

Badges

badges

id UUID PK title description icon condition

Insights

insights

id UUID PK user_id FK title description co2_savings status

Leaderboard

leaderboard_entries

id UUID PK user_id FK green_score rank week

Notifications

notifications

id UUID PK user_id FK type title message read created_at

Activity Logs

activity_logs

id UUID PK user_id FK ip_address device browser location event created_at

---

Refresh Tokens

refresh_tokens

id UUID PK user_id FK token_hash expires_at revoked created_at

---

File Uploads

uploads

id UUID PK user_id FK file_url type size created_at

Entity Relationships

Users │ ├── Carbon Profile ├── Daily Logs ├── Goals ├── Streaks ├── Notifications ├── Badges ├── Leaderboard ├── Uploads └── Activity Logs

---

Authentication System

Authentication Method:

JWT + Refresh Token Architecture

Access Token: 15 minutes

Refresh Token: 30 days

Storage:

Access Token: Memory

Refresh Token: HTTP Only Cookie

Never store tokens in localStorage.

Authentication Flow

Register ↓

Email Verification

↓

Login

↓

Access Token

↓

Refresh Token Rotation

↓

Logout

↓

Token Revocation

Login Methods

1. Email + Password
2. Google OAuth
3. GitHub OAuth

Future:

Apple Login Microsoft Login

Password Security

Algorithm:

Argon2id

Fallback: bcrypt (12 rounds)

Password Rules:

Minimum: 12 characters

Must Include:

Uppercase Lowercase Number Special Character

Password Strength Meter: Frontend

Email Verification Flow

User Registers ↓

Generate Verification Token

↓

Send Email

↓

Verify Email

↓

Activate Account

Token Expiry:

24 Hours

Forgot Password Flow

Request Reset ↓

Generate Token ↓

Email Link ↓

Reset Password ↓

Invalidate Old Sessions

Token Expiry:

15 Minutes

Session Security

Maximum Devices:

5 Sessions

User Dashboard:

Manage Sessions

Displays:

Device Browser Location IP

Logout All Devices: Supported

API Architecture

Pattern:

REST API

Versioning:

/api/v1/

Future:

GraphQL Layer

Authentication APIs

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/logout

POST /api/v1/auth/refresh

POST /api/v1/auth/forgot-password

POST /api/v1/auth/reset-password

POST /api/v1/auth/verify-email

GET /api/v1/auth/me

User APIs

GET /api/v1/users/profile

PATCH /api/v1/users/profile

PATCH /api/v1/users/settings

DELETE /api/v1/users/account

Carbon APIs

POST /api/v1/carbon/onboarding

GET /api/v1/carbon/profile

POST /api/v1/carbon/calculate

GET /api/v1/carbon/history

Tracker APIs

POST /api/v1/tracker/log

GET /api/v1/tracker/logs

GET /api/v1/tracker/streak

GET /api/v1/tracker/badges

Insights APIs

GET /api/v1/insights

POST /api/v1/insights/save

DELETE /api/v1/insights/remove

Goal APIs

POST /api/v1/goals

GET /api/v1/goals

PATCH /api/v1/goals/:id

DELETE /api/v1/goals/:id

Leaderboard APIs

GET /api/v1/leaderboard

GET /api/v1/leaderboard/me

Notifications APIs

GET /api/v1/notifications

PATCH /api/v1/notifications/:id

DELETE /api/v1/notifications/:id

Security Architecture

HTTPS Everywhere

TLS 1.3 only

Password Hashing

Argon2id

Rate Limiting

Login: 5 requests/minute

Register: 3 requests/minute

Password Reset: 2 requests/hour

API: 100 requests/minute

CSRF Protection

Double Submit Cookie Strategy

XSS Protection

Input Sanitization Content Security Policy Escape Output

SQL Injection Prevention

Prisma ORM Parameterized Queries

Security Headers

CSP HSTS X-Frame-Options Referrer Policy X-Content-Type-Options

---

DDoS Protection

Cloudflare Rate Limiting Caching WAF Rules

Secrets Management

AWS Secrets Manager

Never Store:

Passwords API Keys JWT Secrets Database Credentials Inside Code

---

Logging System

Application Logs Error Logs Security Logs Authentication Logs Audit Logs

Stored:

Loki + Grafana

Retention:

90 Days

Monitoring

Sentry: Crash Reporting

Prometheus: Metrics

Grafana: Dashboards

Alerts:

CPU > 80% Memory > 80% Error Rate > 5% Database Latency > 300ms

---

Backup Strategy

PostgreSQL:

Daily Backup 7-Day Point-in-Time Recovery

S3:

Versioning Enabled

Disaster Recovery:

RTO: 30 minutes

RPO: 5 minutes

Future Architecture

AI Carbon Coach Service

Receipt Scanner Service

Carbon Offset Marketplace

Organization Dashboard Service

Wearable Integration Service

Smart Home API Service

Carbon Footprint Public API

Recommended Folder Structure

backend/

src/

modules/ auth/ users/ carbon/ tracker/ goals/ insights/ leaderboard/ notifications/

common/ guards/ interceptors/ middlewares/ filters/ decorators/

config/ database/ redis/ email/ storage/

prisma/ schema.prisma

tests/

docker/

.github/

Production Stack Summary

Frontend: Next.js + TypeScript

Backend: NestJS + TypeScript

Database: PostgreSQL

ORM: Prisma

Cache: Redis

Authentication: JWT + Refresh Tokens + OAuth

Storage: AWS S3

Monitoring: Sentry + Grafana + Prometheus

Deployment: Docker + AWS + Cloudflare + Vercel

Security: Argon2id + HTTPS + CSP + Rate Limiting + WAF

Architecture: Modular Monolith → Microservices Ready